



43. Semi-Supervised Learning

When the data refuses to fully speak, but also refuses to stay silent.

[Previous Section:](#)

[42. Apriori Algorithm](#)

Contents:

[1. Semi-Supervised Learning – The Overview](#)

[2. Semi-Supervised Learning – Label Propagation](#)

[3. Label Propagation with Python](#)

[Summary](#)

[References](#)

This section explores **Semi-Supervised Learning**, a middle ground between Supervised and Unsupervised Learning. Before we define it, we need to understand why it exists in the first place.

- **Unsupervised Learning** focuses on finding structure in data without any labels. It tries to discover patterns on its own—useful when we don't know what we are looking for, or when no labeled data exists.
- **Supervised Learning**, on the other hand, is the “classic” approach. We give the model labeled data, and it learns a direct mapping from inputs to outputs. It is powerful, reliable, and widely used—but it comes with a cost: **it depends heavily on labeled datasets**.

At first, this might not seem like a big issue. We could just ignore the unlabeled part (use it as the testing data) and train on what we have. But then another question appears:

What if the unlabeled data is far larger than the labeled data?

Throwing it away feels wasteful. Using only labeled data feels incomplete. And trying to label everything manually is often expensive, slow, or impossible. This creates a gap between theory and reality.

So instead of choosing between Supervised and Unsupervised Learning, we ask a better question:

Can we use both labeled and unlabeled data together?

This is exactly where Semi-Supervised Learning comes in.

1. Semi-Supervised Learning – The Overview

Semi-Supervised Learning sits in the middle of Supervised and Unsupervised Learning. It is not a separate world of its own, but rather a compromise between two extremes: one where everything is labeled, and one where nothing is.

In this setting, the model is trained using a **small amount of labeled data** (inputs paired with correct outputs) together with a **large amount of unlabeled data** (inputs without labels). Even though most of the dataset has no answers attached, the goal does not change—it is still to build a model that can correctly predict labels for new, unseen inputs.

This becomes especially important in real-world situations, where labeling data is expensive, slow, or requires expert knowledge. In contrast, collecting raw data is often trivial.

For example, it is easy to gather thousands of records containing basic attributes like height, age, or weight. But assigning meaningful labels—such as medical conditions, educational background, or nationality—requires manual verification, expertise, and time. This imbalance creates a natural gap: we have plenty of data, but not enough labeled data.

Semi-Supervised Learning is designed to exploit exactly this gap.

Core Components of Semi-Supervised Learning

This learning setup typically revolves around three key components:

- **Labeled Data:** A small subset of the dataset where each input x is paired with a correct output y . These examples act as the foundation of learning. They tell the model what “correct behavior” looks like.
- **Labels (Ground Truth):** These are the true answers attached to the labeled examples. They serve as anchors that guide the model during training, preventing it from drifting away from meaningful predictions.
- **Unlabeled Data:** A much larger collection of inputs that do not come with answers. At first glance, they seem useless—but they are not. Even without labels, they contain structure: clusters, patterns, and relationships between data points. The model can still learn how the data is organized just by observing how points group together.

To understand this intuitively, imagine you are learning mathematics.

You are first given a small set of solved problems—complete with formulas and step-by-step solutions. These are your labeled examples. They show you what correct reasoning looks like.

Then you receive a much larger set of practice problems, but this time without answers. At first, they feel incomplete. But instead of ignoring them, you start analyzing them. You notice that certain problems look structurally similar. Some follow the same patterns, even if the numbers change.

Gradually, you group similar problems together. Then you use the solved examples to figure out what method applies to each group. Eventually, when you face a new problem in an exam, you don't memorize it—you recognize its structure and know which strategy to apply.

That is exactly what Semi-Supervised Learning is doing.

At its core, this method relies on a simple but powerful assumption:

■ If two data points are close or similar in the input space, they are likely to share the same label.

This assumption allows unlabeled data to contribute meaningful information. Even without explicit answers, it helps reveal how the dataset is structured—how data points cluster, connect, and relate to each other.

A natural question arises: **Why not just ignore unlabeled data and treat it as test data?** The problem is scale and scarcity.

In most real-world datasets, labeled data is rare, while unlabeled data is abundant. If we ignore the unlabeled portion, we force the model to learn from only a small fraction of the available information. This often leads to poor generalization, because the model never sees the full structure of the problem.

Semi-Supervised Learning avoids this limitation by allowing unlabeled data to participate in shaping the model's understanding. Instead of discarding it, we use it to guide learning indirectly, resulting in a system that is both more efficient and more aware of the underlying data distribution.

2. Semi-Supervised Learning – Label Propagation

Label Propagation is one of the most widely used methods in Semi-Supervised Learning. Its core idea is simple, but powerful:

■ Labels from known data points gradually “spread” to nearby unlabeled points based on similarity.

Instead of learning a direct function like traditional supervised models, Label Propagation treats the dataset as a **graph structure**:

- Each data point becomes a **node**
- Similarity between points becomes **edges**
- Labels flow through these edges until the system stabilizes

This follows the same assumption used in semi-supervised learning: **If two points are close in feature space, they are likely to share the same label.**

We use the following dataset:

Point	Feature (x)	Feature (y)	Label
A	1	2	0
B	2	4	?
C	3	2	?
D	4	6	1
E	5	7	?
F	6	1	?
G	7	9	1
H	8	2	?

- Labels: **0 and 1**
- Only A, D, and G are labeled
- The rest are unknown and must be inferred

Step 1: Similarity Measurement (RBF Kernel)

To understand how labels should flow, we first measure similarity between points using the **Radial Basis Function (RBF) kernel**:

$$K(x_i, x_j) = e^{-\frac{\|x_i - x_j\|^2}{2\sigma^2}}$$

Where:

- $\|x_i - x_j\|$ = Euclidean distance
- σ = bandwidth parameter (here: $\sigma = 1.0$)

Example Calculations:

Similarity between A and B $\rightarrow A = (1, 2), B = (2, 4)$

Euclidean distance:

$$|A - B| = \sqrt{(1 - 2)^2 + (2 - 4)^2} = \sqrt{5} \approx 2.236$$

RBF similarity:

$$K(A, B) = e^{-\frac{5}{2}} \approx 0.0821$$

Similarity between A and C $\rightarrow A = (1, 2), C = (3, 2)$

Euclidean distance:

$$|A - C| = \sqrt{(1 - 3)^2 + (2 - 2)^2} = 2$$

RBF similarity:

$$K(A, C) = e^{-2} \approx 0.1353$$

Step 2: Similarity Matrix (W)

After computing all pairwise similarities, we obtain the full **8 × 8 similarity matrix**:

	A	B	C	D	E	F	G
A	1	0.082085	0.135335	3.73E-06	1.25E-09	2.26E-06	3.48E-16
B	0.082085	1	0.082085	0.018315	0.000123	3.73E-06	1.38E-11

	A	B	C	D	E	F	G
C	0.135335	0.082085	1	0.000203	5.04E-07	0.006737	7.68E-15
D	3.73E-06	0.018315	0.000203	1	0.367879	0.606531	0.000123
E	1.25E-09	0.000123	5.04E-07	0.367879	1	9.23E-09	0.018315
F	2.26E-06	3.73E-06	0.006737	0.606531	9.23E-09	1	7.68E-15
G	3.48E-19	1.38E-11	7.68E-15	0.000123	0.018315	7.68E-15	1
H	2.29E-11	2.06E-09	3.73E-06	1.12E-07	4.13E-08	0.082085	1.38E-11

Observations

- Diagonal = 1 → perfect self-similarity
- Large distances → similarity → ~0
- Example:
 - $A-B \approx 0.082 \rightarrow$ moderately similar
 - $A-C \approx 0.135 \rightarrow$ slightly more similar
 - $A-D \approx 3.73 \times 10^{-6} \rightarrow$ almost unrelated

This matrix is essentially a **weighted graph**:

- Nodes = data points
- Edges = similarity
- Weights = strength of connection

Step 3: Label Representation

Instead of single labels, we use probability vectors:

Point	Y_0	Y_1	Label
A	1	0	0
B	0	0	?
C	0	0	?
D	0	1	1
E	0	0	?
F	0	0	?
G	0	1	1
H	0	0	?

- Y_0 = probability of class 0
- Y_1 = probability of class 1
- Unlabeled points start at $[0, 0]$

Step 4: Label Propagation Rule

The update equation is:

$$F_{t+1} = \alpha W F_t + (1 - \alpha) Y$$

Where:

- W = similarity matrix
- F_t = label scores at iteration t
- Y = initial labels

- $\alpha \in [0.8, 0.99]$ (here: 0.9)

Step 5: First Iteration Result

After one propagation step:

Point	Class 0	Class 1	Predicted
A	1	0.195678	0 (correct)
B	0.0738765	1.0738765	1
C	0.12180177	1.0738765	1
D	3.357E-06	0.0166672	1 (correct)
E	1.125E-09	0.00011152	1
F	2.034E-06	0.00606751	1
G	3.138E-19	1.250E-11	1 (correct)
H	2.060E-11	3.355E-06	1

Label decision rule: $\hat{y}_i = \arg \max(F_i)$

Step 6: Interpretation of Propagation

Even after one iteration, we already see the structure forming:

- Class **1 spreads strongly** around D and G
- Points near D and G gradually shift toward class 1
- A retains class 0 influence but weakly spreads
- Unlabeled points begin aligning with their nearest labeled neighbors

Step 7: Iterative Convergence

This process is repeated:

$$F_{t+1} = \alpha W F_t + (1 - \alpha) Y$$

Until:

- Values stop changing
- The system reaches **stability (convergence)**

At convergence:

- Each point has a stable probability distribution
- Final labels are assigned using argmax

Label Propagation is not "learning rules" in the traditional sense. It is something simpler—but deeper:

It lets labels behave like energy flowing through a network, spreading from certainty into uncertainty, until the entire structure becomes consistent.

It is learning through **structure, not supervision alone**.

3. Label Propagation with Python

To make Semi-Supervised Learning more concrete, we apply **Label Propagation** on a real dataset: the **Breast Cancer Wisconsin dataset**:

[breast_cancer_wisconsin.csv](#)

This example shows how the algorithm behaves when most labels are hidden and only a small portion of labeled data is available.

- **Step 1: Load and Prepare the Dataset**

```
import pandas as pd

# Load dataset
df = pd.read_csv('breast_cancer_wisconsin.csv')

target = 'diagnosis'
unique_labels = df[target].unique()

# Convert categorical labels into numeric form
df[target] = df[target].replace({unique_labels[i]: i for i in range(len(unique_labels))})
```

- **Step 2: Simulate Semi-Supervised Learning**

Even though the dataset is fully labeled, we artificially simulate a semi-supervised scenario.

```
# Copy original labels
y_unlabeled = df[target].copy()

# Randomly select 80% of samples to hide labels
unlabeled_idx = df.sample(frac=0.8, random_state=42).index

# Replace their labels with -1 (unlabeled marker)
y_unlabeled.loc[unlabeled_idx] = -1
```

Now the dataset contains: 20% labeled data; 80% unlabeled data (labeled as -1)

- **Step 3: Prepare Features and Scaling**

```
from sklearn.preprocessing import StandardScaler

# Features and labels
X = df.drop(columns=[target]).values
y_semi_array = y_unlabeled.values

# Standardization (important for distance-based methods)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

- **Step 4: Train Label Propagation Model**

We use **RBF kernel-based Label Propagation**:

```
from sklearn.semi_supervised import LabelPropagation

# Initialize model
model = LabelPropagation(kernel='rbf', gamma=20, max_iter=1000)
# Train model using partially labeled data
model.fit(X_scaled, y_semi_array)
```

```
# Final predicted labels (for all samples)
y_pred = model.transduction_
```

- **Step 5: Evaluation**

After propagation, we compare predictions with true labels:

```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score, adjusted_rand_score

y_true = df[target].values

print("Confusion Matrix:\n", confusion_matrix(y_true, y_pred))
print("Accuracy:", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred))
print("Recall:", recall_score(y_true, y_pred))
print("F1 Score:", f1_score(y_true, y_pred))
print("ARI:", adjusted_rand_score(y_true, y_pred))
```

Confusion Matrix:

```
[[203  9]
```

```
[ 23 334]]
```

Accuracy: 0.9437609841827768

Precision: 0.9737609329446064

Recall: 0.9355742296918768

F1 Score: 0.9542857142857143

ARI: 0.7867736023958896

- **Step 6: Visualization (Before vs After Propagation)**

We can visually observe how labels spread across the dataset:

```
import matplotlib.pyplot as plt
import numpy as np

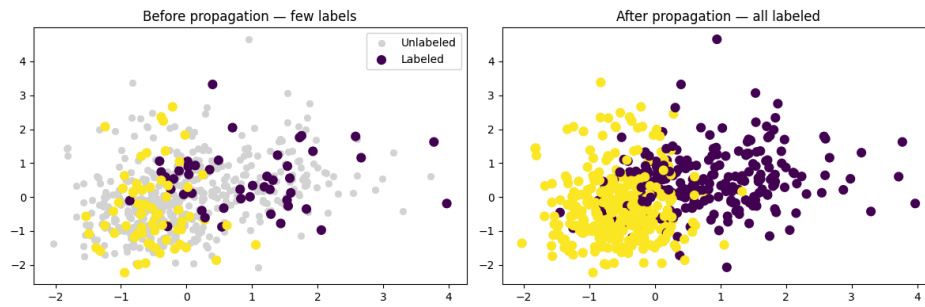
# True labels for colored plot (before propagation)
y = df[target].values
mask = y_semi_array != -1 # labeled points

fig, ax = plt.subplots(1, 2, figsize=(12, 4))

# Before propagation
ax[0].scatter(X_scaled[:, 0], X_scaled[:, 1], c='lightgray', s=30, label='Unlabeled')
ax[0].scatter(X_scaled[mask, 0], X_scaled[mask, 1], c=y[mask], cmap='viridis', s=60, label='Labeled')
ax[0].set_title("Before propagation – few labels")
ax[0].legend()

# After propagation
ax[1].scatter(X_scaled[:, 0], X_scaled[:, 1], c=y_pred, cmap='viridis', s=60)
ax[1].set_title("After propagation – all labeled")

plt.tight_layout()
plt.show()
```



Before propagation:

- Only a small fraction of points are labeled
- Most of the dataset is unknown

After propagation:

- Every point receives a predicted label
- Labels are influenced by nearby labeled samples
- The dataset becomes fully labeled through similarity structure

This demonstrates the key idea:

Even without explicit supervision, structure in the data can guide learning.

Summary

Semi-Supervised Learning, especially through **Label Propagation**, shows how powerful unlabeled data can be when combined with a small amount of supervision.

Instead of ignoring missing labels, the algorithm treats the dataset as a connected structure where information can flow between similar points. This allows the model to expand knowledge from a few labeled examples to the entire dataset.

In this section, we only explored **Label Propagation**, which is one of the simplest and most intuitive semi-supervised methods. However, it is not the only approach.

There are many other techniques such as:

- Self-training methods
- Consistency regularization
- Graph-based neural approaches

Each of these builds on the same core idea:

Unlabeled data is not useless—it is hidden structure waiting to be used.

So far, Label Propagation gives us the first clear intuition of how this concept works in practice.

References

<https://www.ibm.com/think/topics/semi-supervised-learning>

<https://www.geeksforgeeks.org/machine-learning/ml-semi-supervised-learning/>

<https://machinelearningmastery.com/semi-supervised-learning-with-label-propagation/>

<https://www.youtube.com/watch?v=C3Lr6Waw66g>

Next Section:

 **44. Leveraging Pre-Trained Models**